

Les Autoloaders en PHP

1. ? Pourquoi un autoloader ?

En PHP, pour utiliser une classe, il faut charger son fichier :

```
require 'classes/Utilisateur.php';
```

👉 Cela devient vite **ingérable** quand il y a des dizaines de classes.

➡ **Un autoloader** permet de **charger automatiquement les classes** dès qu'on essaie de les instancier.

2. 🔄 Autoloading manuel (PHP natif)

Depuis PHP 5, on peut enregistrer une fonction d'autochargement :

```
spl_autoload_register(function ($classe) {  
    require 'classes/' . $classe . '.php';  
});
```

Exemple :

```
// Fichier : classes/Utilisateur.php  
class Utilisateur {  
    public function hello() {  
        echo "Bonjour !";  
    }  
}  
  
// Fichier : index.php  
$u = new Utilisateur(); // La classe est automatiquement chargée
```

⚠ **Problème** : cela ne fonctionne pas bien avec les namespaces, ou les conventions de dossier avancées.

3. 📖 Norme PSR-4 (moderne)

PSR-4 est une norme PHP qui définit une **correspondance entre les namespaces et l'arborescence de fichiers**.

[Specs PSR-4](#)

Exemple :

```
// Fichier : src/Model/User.php
namespace App\Model;

class User { }
```

Avec PSR-4, le chemin `src/Model/User.php` correspond au namespace `App\Model\User`.

4. Autoloader avec Composer (recommandé)

Étapes :

✓ 1. Initialiser Composer

```
composer init
```

✓ 2. Ajouter l'autoload

Dans le `composer.json` :

```
"autoload": {
  "psr-4": {
    "App\\": "src/"
  }
}
```

✓ 3. Créer la classe

```
// src/Model/User.php
namespace App\Model;

class User {
    public function hello() {
        echo "Salut de User !";
    }
}
```

✓ 4. Générer l'autoloader

```
composer dump-autoload
```

✓ 5. Utiliser dans index.php

```
require 'vendor/autoload.php';

use App\Model\User;

$u = new User();
$u->hello();
```

5. 📁 Structure recommandée

```
/mon-projet
├── src/
│   └── Model/
│       └── User.php
├── vendor/
├── composer.json
└── index.php
```

6. 🧠 Avantages d'un autoloader Composer

✓ Avantage	Explication
Standardisé	Compatible avec les frameworks et bibliothèques modernes
Évolutif	Plus besoin de gérer manuellement les require
Compatible PSR-4	Organisation claire du code par namespace
Intégré à Composer	Centralisation de la config des dépendances

7. 💡 Bonnes pratiques

- **Respecter PSR-4** pour vos namespaces et chemins
- Toujours **utiliser vendor/autoload.php** comme point d'entrée
- Nommer les fichiers et classes avec la **même casse**